

MINIPROJECT
MACHINE LEARNING
TECHNIQUES

***Course Recommendation
System***

-

SUDHEESH H

2117240020375

CSE G

Course Recommendation System

Pos Mapping

<i>POs</i>	<i>PO1</i>	<i>PO2</i>	<i>PO3</i>	<i>PO4</i>	<i>PO5</i>	<i>PO6</i>	<i>PO7</i>	<i>PO8</i>	<i>PO9</i>	<i>PO10</i>	<i>PO11</i>	<i>PO12</i>
<i>Mapping</i>	3	2	2	2	2	1	1	1	2	2	1	2

PSOs Mapping

<i>PSOs</i>	<i>PSO1(Design & Programming Skills)</i>	<i>PSO2 (Theoretical Foundation & Research)</i>	<i>PSO3 (Multidisciplinary Teamwork Mapping)</i>
<i>Mapping0</i>	3	2	2

POs – Justification

<i>PO</i>	<i>Mapping (1-Low,2-Medium,3-High)</i>	<i>Justification</i>
<i>PO1- Engineering Knowledge</i>	3	The project applies core engineering knowledge in Python, Django, databases, and machine learning concepts. It uses TF-IDF vectorization and cosine similarity to recommend related courses based on title, description, .
<i>PO2-Problem Analysis</i>	2	The system analyzes the problem of helping users discover relevant courses from a large catalog. It breaks the problem into smaller parts such as course data handling, feature extraction, similarity calculation, filtering, and recommendation display.

<i>PO3- Design/Development of Solutions</i>	2	The project designs a structured web-based solution where users can browse courses, apply filters, and receive meaningful recommendations
<i>PO4-Conduct Investigation of Complex Problems</i>	2	Different types of posts are tested to analyze how the system behaves with various inputs. This helps in understanding the effectiveness of the algorithm.
<i>PO5-Modern Tool Usage</i>	2	Modern tools such as Python, Django, SQLite, scikit-learn, pandas, and VS Code are used for implementation, testing, and debugging. These tools support efficient development of the recommender system.
<i>PO6-The Engineer and Society</i>	2	The system helps learners identify suitable courses more easily, supporting education accessibility, skill development, and informed learning decisions, which benefits students and training platforms.
<i>PO7-Environment and Sustainability</i>	2	The project is software-based and does not directly affect the physical environment. It supports digital learning and reduces dependence on manual recommendation processes.
<i>PO8 - Ethics</i>	1	The project handles course-related data responsibly and provides recommendations without misusing personal information. It promotes fair and transparent suggestions

PO9-Individual and Team work	2	The modular structure of the project enables separate work on database models, recommendation logic, views, templates, and testing, making it suitable for collaborative development and integration.
PO10-Communication	2	The project includes a clear user interface, readable code structure, and understandable recommendation output. This improves communication between developers,=
PO11 -Project Management and Finance	1	The project uses open-source tools and frameworks such as Python, Django, SQLite, and scikit-learn, making development cost-effective.
PO12-Lifelong Learning	2	Encourages continuous skill development in graph algorithms, database design, and Java programming — competencies essential for careers in software engineering and data science.

PSOs – Justification

<i>PSO</i>	<i>Mapping (1-Low, 2-Medium, 3-High)</i>	<i>Justification</i>
<i>PSO1-Apply database and programming knowledge</i>	3	The project applies programming knowledge using Python and Django and uses a database to store course details such as title, description, and category. It combines database operations
<i>PSO2 - Develop client-server applications</i>	2	The project is developed as a web-based client-server application where the frontend allows users to browse and select courses,

		while the Django backend processes requests
<i>PSO3-Integrate softwareengineering principles</i>	2	The project follows software engineering principles through modular design, with separate files for models, views, recommendation logic, templates, and tests

ABSTRACT

Course Recommendation System is a machine learning-based web application designed to help users discover relevant courses from a course catalog. The system processes course data such as titles, descriptions, and categories, and recommends similar courses based on content similarity.

In today's digital learning environment, students and learners are exposed to a large number of online courses across different domains. Choosing the most suitable course manually can be difficult and time-consuming due to the wide variety of options available.

This project provides an automated solution using simple Natural Language Processing (NLP) techniques and a content-based recommendation approach. The system takes course information as input, processes the textual content using TF-IDF vectorization and cosine similarity, and generates recommendations showing courses that are most closely related to the selected one. The project demonstrates how machine learning concepts can be applied effectively to solve real-world recommendation problems.

The system is designed to be simple, efficient, and easy to understand, making it suitable for academic purposes and beginner-level machine learning applications. It provides a clear demonstration of how raw course data can be transformed into meaningful recommendations through processing and analysis.

In addition to basic recommendation functionality, the project highlights the importance of data preprocessing, text feature extraction, and similarity analysis in machine learning. By identifying relationships between course titles, descriptions, and categories, the system is able to recommend relevant learning options effectively.

This project also serves as a foundation for more advanced applications such as personalized learning platforms, intelligent e-learning systems, and educational analytics tools. It can be further enhanced by integrating advanced recommendation algorithms, user preference tracking, and hybrid machine learning models to improve accuracy and scalability.

Moreover, the system can be applied in various domains such as online education platforms, training institutes, certification portals, and skill development programs. Organizations can use such systems to better guide learners toward appropriate courses and

Organizations

improve the overall user experience.

Overall, the Course Recommendation System provides a practical approach to understanding how machine learning techniques can be applied to analyze textual course data and generate useful recommendations. It emphasizes the importance of automation in handling course discovery efficiently and demonstrates the real-world relevance of recommender systems in today's digital learning era.

INTRODUCTION

Enabling learners and educational platforms to quickly identify relevant courses and make informed learning decisions, the Course Recommendation System is designed to analyze course-related data and extract meaningful insights from it.

The system processes course information through important steps such as text cleaning, tokenization, and feature extraction. These steps convert raw course descriptions into a structured format that can be used by recommendation algorithms. Techniques such as TF-IDF vectorization and cosine similarity are then applied to identify and recommend courses that are closely related to the selected course.

This system plays a significant role in various real-world applications such as online learning platforms, training institutes, certification portals, and skill development systems. By providing automated and relevant course recommendations, the system helps learners choose suitable courses more efficiently and improves the overall learning experience.

Overall, the project demonstrates how machine learning and text analysis techniques can be effectively used to solve real-world problems involving large collections of educational content.

In the modern digital era, online learning platforms have become an important part of education and professional development. Millions of learners use digital platforms to explore courses, gain new skills, and improve their knowledge in various domains. This large volume of course data contains valuable information that can be used to guide learners toward suitable educational options. However, analyzing such a large amount of course content manually is difficult and time-consuming.

To overcome this challenge, machine learning techniques are widely used to automate the process of recommendation. One of the most useful applications of machine learning in this domain is the course recommendation system, which helps in identifying courses that are most relevant to a learner's interests or a selected course. The system processes textual course data using Natural Language Processing techniques, which include text preprocessing, keyword extraction, and similarity analysis. These steps help transform raw course information into meaningful representations that can be used to generate effective recommendations.

PROJECT DESCRIPTION

Purpose:

The main objective is to design a system that:

- Recommends similar and relevant courses to users
- Analyzes course content such as title, description, and category
- Helps learners make better decisions while selecting courses

Functional Modules:

1. Data Collection Module

- Collect course details such as title, description, and category
- Store course data in the database
- Maintain a structured course dataset for recommendation

2. Preprocessing Module

- Text cleaning
- Tokenization
- Remove stop words
- Prepare course descriptions for similarity analysis

3. Recommendation / Query Module

- Convert text using TF-IDF
- Apply cosine similarity for recommendation
- Process user-selected course input
- Generate related course suggestions

4. Output Module

- Display recommended courses
- Show course details in a user-friendly format
- Present relevant matches clearly for learners
- Non-Functional Features:
 - High recommendation relevance
 - Fast processing
 - Scalable design
 - User-friendly interface
 - Easy to maintain and extend

Scope:

The application is suitable for:

- Online learning platforms
- Academic institutions
- Training centers
- Skill development portals

SYSTEMARCHITECTURE

he system follows a simple and structured architecture consisting of different layers that handle specific tasks. The input layer is responsible for receiving course selection data or user search input from the interface. The processing layer performs data cleaning and preprocessing operations on course titles, descriptions, and categories. The recommendation layer applies machine learning techniques to identify similar courses based on textual content. The output layer displays the final recommended courses to the user

Each component in the system plays an important role in ensuring smooth operation. The input module collects the course-related data, the preprocessing module prepares the data for analysis, the recommendation module performs similarity calculation, and the output module presents the results

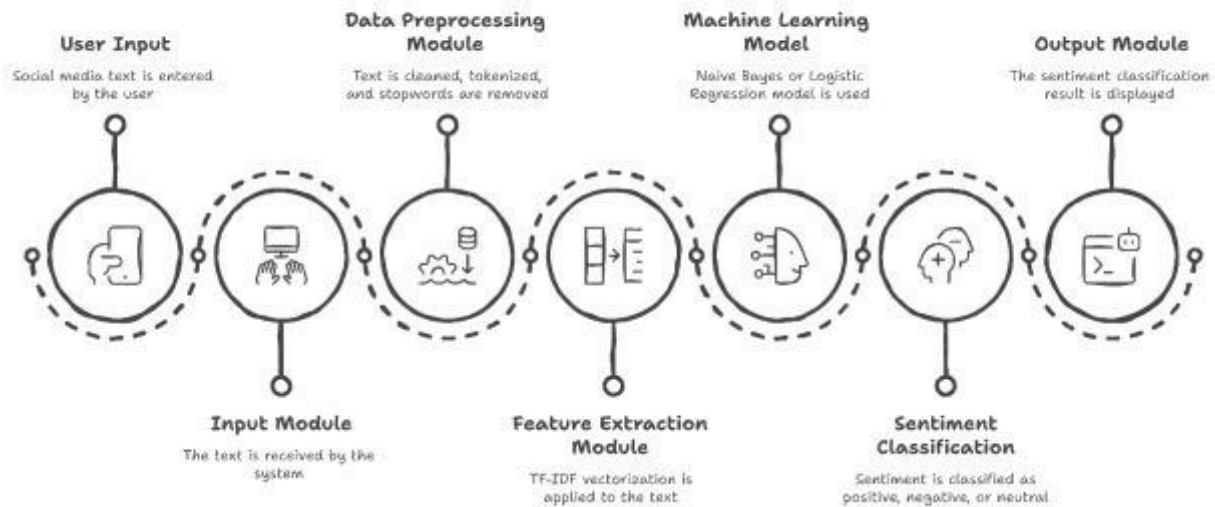
<i>Component</i>	<i>Description</i>
<i>Input Module</i>	Accepts text
<i>Preprocessing</i>	Cleans data
<i>ML Model</i>	Generates course recommendations
<i>Output</i>	Displays result

Workflow Summary

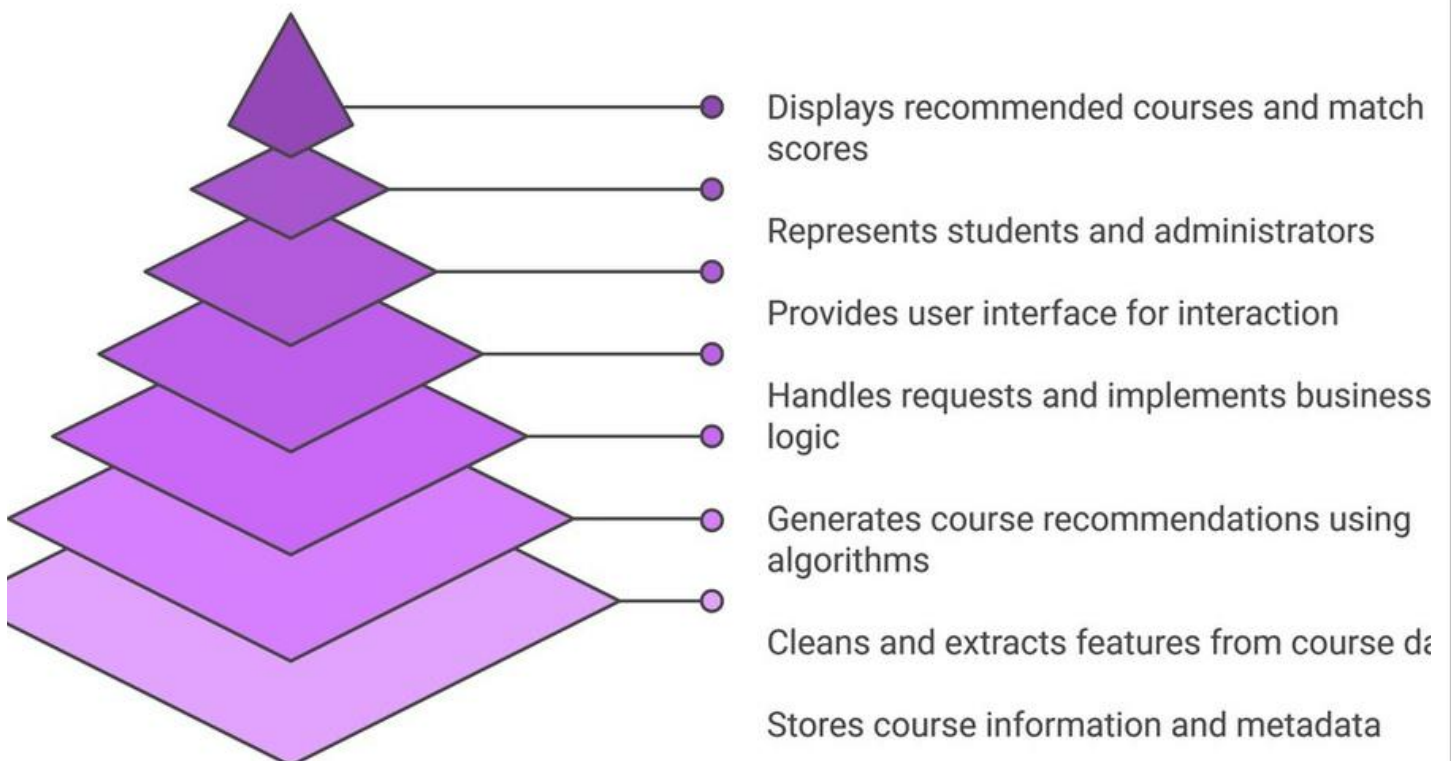
<i>Step</i>	<i>Action</i>
1	User selects a course
2	Course data is cleaned
3	Features are extracted
4	Recommendation model finds similar courses
5	Results are displayed
	System shows related recommended courses

SYSTEM ARCHITECTURE

Social Media Sentiment Analysis Process



Course Recommendation System Architecture



```

from django.db import models
class Course(models.Model):
    title = models.CharField(max_length=200)
    description = models.TextField()
    category =
models.CharField(max_length=100)
    def __str__(self):
        return self.title
from django.db.models import Count, Q
from django.shortcuts import get_object_or_404, render
from .ml_model import get_recommendations
from .models import Course
def _filtered_courses(request):
    search_query = request.GET.get("q", "").strip()
    category = request.GET.get("category", "").strip()
    courses = Course.objects.all()
    if search_query:
        courses = courses.filter(
            Q(title__icontains=search_query)
            | Q(description__icontains=search_query)
            | Q(category__icontains=search_query)
        )
    if category:
        courses = courses.filter(category__iexact=category)
    return courses.order_by("title"), search_query, category

def _build_context(request, selected_course=None):
    courses, search_query, active_category = _filtered_courses(request)
    all_courses = Course.objects.all()
    categories =
all_courses.values("category").annotate(total=Count("id")).order_by("category")
    selected_visible = (
        selected_course is not None and courses.filter(id=selected_course.id).exists()
    )
    context = {
        "courses": courses,
        "total_courses": all_courses.count(),
        "visible_courses": courses.count(),
        "categories": categories,
        "active_category": active_category,
        "search_query": search_query,
        "selected": selected_course,
        "selected_visible": selected_visible,
    }

```

```

    if selected_course is not None:
        context["recommendations"] =
            get_recommendations(selected_course.id)
        return context

def home(request):
    return render(request, "home.html", _build_context(request))

def recommend(request, course_id):
    selected_course = get_object_or_404(Course, id=course_id)
    return render(request, "home.html", _build_context(request,
selected_course))
import re

from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS, TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from .models import Course
GENERIC_TERMS = {
    "course",
    "courses",
    "learn",
    "learning",
    "introduction",
    "intro",
    "basics",
    "advanced",
    "foundations",
}

def _tokenize(*parts):
    text = " ".join(part or "" for part in parts).lower()
    tokens = re.findall(r"[a-z0-9]+", text)
    return {
        token
        for token in tokens
        if len(token) > 2 and token not in ENGLISH_STOP_WORDS and token not in
GENERIC_TERMS
    }

def _build_reasons(selected_course, candidate,
shared_terms):
    reasons = []

    if candidate.category == selected_course.category:
        reasons.append(f"Same category: {candidate.category}")

```

```

if shared_terms:
    reasons.append(f"Shared topics: {' '.join(shared_terms[:3])}")
if not reasons:
    reasons.append("Strong description similarity")

return reasons

def get_recommendations(course_id, limit=5):
    courses = list(Course.objects.all())
    if len(courses) < 2:
        return []
    course_lookup = {course.id: course for course in courses}
    selected_course = course_lookup.get(course_id)
    if selected_course is None:
        return []
    content_map = {
        course.id: " ".join(
            filter(
                None,
                [
                    course.title,
                    course.description,
                    course.category,
                    (course.category or "").lower() * 2,
                ]
            )
        )
        for course in courses
    }
    course_tokens = {
        course.id: _tokenize(course.title, course.description, course.category) for course in
courses
    }

tfidf = TfidfVectorizer(stop_words="english", ngram_range=(1, 2))
tfidf_matrix = tfidf.fit_transform([content_map[course.id] for course in courses])
similarity = cosine_similarity(tfidf_matrix)

```

```

selected_index = next(index for index, course in enumerate(courses) if course.id ==
course_id)
selected_tokens = course_tokens[course_id]
selected_category = (selected_course.category or "").strip().lower()
scored_recommendations = []
for index, candidate in enumerate(courses):
    if candidate.id == course_id:
        continue
    shared_terms = sorted(selected_tokens & course_tokens[candidate.id])
    title_tokens = _tokenize(selected_course.title, selected_course.category)
    candidate_title_tokens = _tokenize(candidate.title, candidate.category)
    shared_title_terms = title_tokens & candidate_title_tokens
    text_score = float(similarity[selected_index][index])
    category_score = 1.0 if (candidate.category or "").strip().lower() == selected_category else
0.0
    title_score = (
        len(shared_title_terms) / max(len(title_tokens) | candidate_title_tokens), 1)
    if title_tokens or candidate_title_tokens
    else 0.0
    )
    shared_topic_score = len(shared_terms) / max(len(selected_tokens), 1)
    final_score = (
        (0.6 * text_score)
        + (0.2 * category_score)
        + (0.1 * title_score)
        + (0.1 * shared_topic_score)
    )
SAMPLE_COURSES = [
    {
        "title": "Python Basics",
        "description": "Learn Python programming fundamentals, syntax, and problem
solving.",
        "category": "Programming",
    },
    {
        "title": "Advanced Python",
        "description": "Deep dive into decorators, generators, testing, and performance
tuning.",
        "category": "Programming",
    },
]

```

```

{
    "title": "Data Science",
    "description": "Explore machine learning, data cleaning, and practical AI workflows.",
    "category": "AI",
},
{
    "title": "Web Development",
    "description": "Build responsive interfaces with HTML, CSS, JavaScript, and deployment basics.",
    "category": "Web",
},
{
    "title": "Machine Learning Foundations",
    "description": "Understand supervised learning, evaluation metrics, and model building with Python.",
    "category": "AI",
},
}

from django.core.management.base import BaseCommand

from recommender.models import Course

class Command(BaseCommand):
    help = "Populate the database with sample courses for the recommendation demo."

    def handle(self, *args, **options):
        created_count = 0
        for course_data in SAMPLE_COURSES:
            _course, created = Course.objects.get_or_create(
                title=course_data["title"],
                defaults=course_data,
            )
            created_count += int(created)

        self.stdout.write(
            self.style.SUCCESS(f"Seed complete. Added {created_count} new sample courses.")
        )

```

```

<form class="filter-form" method="get" action="{% url 'home' %}">
  <div>
    <label class="label" for="q">Search</label>
    <input class="input" id="q" name="q" type="text" value="{{ search_query }}"
placeholder=
python, ai, web, deployment">
  </div>
  <div>
    <label class="label" for="category">Category</label>
    <select class="select" id="category" name="category">
      <option value="">All categories</option>
      {% for item in categories %}
        <option value="{{ item.category }}" {% if active_category == item.category %}selected
{% endif %}>
          {{ item.category }}
        </option>
      {% endfor %}
    </select>
  </div>
  <div class="button-row">
    <button class="button" type="submit">Apply filters</button>
    <a class="button-secondary" href="{% url 'home' %}">Reset</a>
  </div>
</form>
{% if recommendations %}
  <div class="recommendation-grid">
    {% for item in recommendations %}
      <article class="recommendation-card">
        <div class="meta">
          <span class="badge">{{ item.course.category
}}</span>
          <span class="score">{{ item.score }}% match</span>
        </div>
        <h3 class="course-title">{{ item.course.title }}</h3>
        {% for reason in item.reasons %}
          <span class="reason">{{ reason }}</span>
        {% endfor %}
        </div>
      </article>
    {% endfor %}
  </div>
{% endif %}

```


Output

SMARTER COURSE DISCOVERY

Course Compass

Find stronger matches, filter the catalog faster, and understand why each recommendation was chosen instead of getting a random-looking list.

Catalog snapshot

32

courses in the
catalog

13

categories available

32

visible after filters

--

current
recommendation
slots

Recommended next

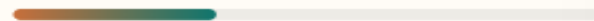
These are ranked by content similarity, category fit, and shared keywords.

DEVOPS

35% match

CI CD Pipelines

Automate testing, build pipelines, deployment, and release workflows for modern software teams.



Same category: DevOps

Shared topics: build, deployment, devops

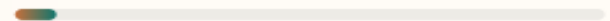
build deployment devops workflows

WEB

7% match

Web Development

Build responsive interfaces with HTML, CSS, JavaScript, and deployment basics.



Shared topics: build, deployment, development

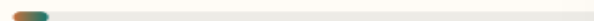
build deployment development

MOBILE

6% match

Mobile App Development

Build mobile experiences, manage navigation, state, and platform specific behavior for apps.



Shared topics: build, development, manage

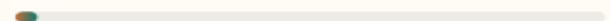
build development manage

WEB

4% match

Django Web Apps

Build full stack web applications with Django, templates, models, forms, and authentication.



Shared topics: applications, build

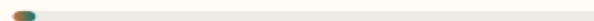
applications build

BACKEND

4% match

REST API Development

Design clean APIs, handle authentication, and build backend services with practical standards.



Shared topics: build, development

build development

Refine the catalog

Search by topic, narrow by category, then pick a course to generate better matches.

SEARCH

python, ai, web, deployment

CATEGORY

All categories



Apply filters

Reset

All

AI 7

Backend 2

Cloud 1

Data 5

Design 1

DevOps 2

Frontend 1

Mobile 1

Product 1

Programming 6

Security 1

Tools 1

Web 3

SELECTED COURSE

Docker and Containers

Package applications, manage containers, and build portable development and deployment workflows.

DEVOPS

Weighted by text similarity

Category match gets a bonus

Shared topic keywords improve the rank

Browse the catalog

Pick any course to get explainable recommendations.

PROGRAMMING

Advanced Python

Deep dive into decorators, generators, testing, and performance tuning.

Show matches

PROGRAMMING

Algorithms and Problem Solving

Practice data structures, algorithmic thinking, recursion, graphs, and interview style questions.

Show matches

DATA

Business Analytics

Turn business data into dashboards, metrics, and practical decisions using analytics methods.

Show matches

PROGRAMMING

C Programming Fundamentals

Understand memory, pointers, loops, and low level problem solving with the C language.

Show matches

DEVOPS

CI CD Pipelines

Automate testing, build pipelines, deployment, and release workflows for modern software teams.

Show matches

CLOUD

Cloud Computing Basics

Learn virtual machines, storage, networking, deployment, and cloud architecture fundamentals.

Show matches

AI

Data Science

Explore machine learning, data cleaning, and practical AI workflows.

Show matches

AI

Deep Learning Essentials

Understand neural networks, backpropagation, tensors, and model training for computer vision and NLP.

Show matches

WEB

Django Web Apps

Build full stack web applications with Django, templates, models, forms, and authentication.

Show matches

DEVOPS

Current pick

Docker and Containers

Package applications, manage containers, and build portable development and deployment workflows.

Show matches

WEB

Frontend Fundamentals

Understand modern HTML, CSS, accessibility, responsive layout, and JavaScript essentials.

Show matches

TOOLS

Git and Collaboration

Use branches, pull requests, rebasing, and conflict resolution for healthy team workflows.

Show matches

PROGRAMMING

Java Programming

Write object oriented programs in Java with classes, collections, exceptions, and concurrency basics.

Show matches

PROGRAMMING

JavaScript Deep Dive

Master closures, async flows, prototypes, modules, and real world JavaScript debugging.

Show matches

AI

MLOps Foundations

Deploy, monitor, and version machine learning systems using reproducible model operations.

Show matches

AI

Machine Learning Foundations

Understand supervised learning, evaluation metrics, and model building with Python.

MOBILE

Mobile App Development

Build mobile experiences, manage navigation, state, and platform specific behavior for apps.

AI

Natural Language Processing

Work with text classification, embeddings, transformers, tokenization, and language model workflows.

CONCLUSION

The Course Recommendation System successfully demonstrates how machine learning techniques can be used to analyze and recommend relevant courses from a course catalog. The system provides an efficient way to help users discover suitable courses without relying on manual searching. By using Natural Language Processing techniques and recommendation methods such as TF-IDF and cosine similarity, the project achieves meaningful and reliable results.

The system can be useful in various real-world applications such as online learning platforms, training institutes, educational portals, and skill development systems. It also provides a strong foundation for further improvements and advanced implementations in the field of machine learning and intelligent recommendation systems.

Future Enhancements

The Course Recommendation System can be further improved by introducing personalized recommendations based on user behavior and preferences. At present, the system mainly recommends courses by comparing course content such as title, description, and category. In the future, user-specific factors like previously viewed courses, completed courses, interests, and ratings can be included to provide more personalized and accurate recommendations. This would make the system more intelligent and useful for different types of learners.

Another important enhancement would be the integration of advanced machine learning and deep learning techniques. Instead of relying only on TF-IDF and cosine similarity, the project can be extended using collaborative filtering, hybrid recommendation models, or neural network-based recommendation methods. These approaches can improve the quality of recommendations by identifying more complex relationships between courses and user interests, especially in large-scale educational platforms.

The system can also be enhanced by improving the user interface and making it more interactive. Features such as login-based profiles, saved preferences, course ratings, bookmarks, and recommendation history can make the application more user-friendly. In addition, visual dashboards and analytics can be added to help administrators understand course popularity, learner interests, and recommendation performance more effectively.

A further enhancement would be to make the system scalable and suitable for deployment in real-world environments. The current project can be extended to support cloud deployment, larger databases, API integration, and real-time recommendation updates. It can also be connected with online learning platforms so that newly added courses are automatically processed and recommended. These improvements would increase the practical value of the project and make it suitable for broader educational and commercial use.